

Azure Service Bus – Connection Strings

1. What is a Connection String?

A **connection string** is a string containing the information and credentials required by an application to connect to Azure Service Bus.

It is used by an application to authenticate and establish a connection with:

- A specific Queue
- A specific Topic
- An entire Service Bus Namespace

Simple idea:

```
Application
  |
  | Connection String
  v
Azure Service Bus
```

2. Shared Access Policies

Azure Service Bus provides **Shared Access Policies** to control access to queues, topics, and namespaces.

A Shared Access Policy defines what operations an application is allowed to perform.

Common permissions are:

Send

Allows the application to **send messages**.

```
Application
  |
  | Send Message
  v
Service Bus Queue
```

Listen

Allows the application to **receive/consume messages**.

```
Service Bus Queue
  |
  | Receive Message
  v
Application
```

Manage

Provides both **Send** and **Listen** permissions, along with management-related permissions.

In the example from the video, a policy named:

```
myQueuePolicy
```

was created with **Manage** permission.

3. Creating a Connection String for a Queue

Suppose we have:

```
Service Bus Namespace
  |
  +---- Queue: OrderQueue
```

If our application only needs to work with `OrderQueue`, we can create a Shared Access Policy specifically for that queue.

Steps

Go to:

```
Azure Portal
  ↓
Service Bus Namespace
  ↓
Queue
  ↓
```

Settings



Shared access policies

Then:

1. Click **Add**
2. Provide a policy name
3. Select the required permissions
4. Create the policy
5. Open the newly created policy
6. The connection information will be available there

The policy provides:

- Primary connection string
- Secondary connection string
- Primary key
- Secondary key

4. Primary and Secondary Connection Strings

When a Shared Access Policy is created, Azure provides:

Primary Connection String
Secondary Connection String

Primary Key
Secondary Key

Why are there two keys?

The primary and secondary keys allow you to perform **credential rotation** without unnecessary downtime.

For example:

Application
|
| Primary Key
v
Azure Service Bus

When the primary key needs to be rotated:

```
Application
  |
  | Secondary Key
  v
Azure Service Bus
```

The primary key can then be regenerated safely.

This is useful for security and key rotation.

5. Queue-Level Connection String

A **queue-level connection string** is associated with a Shared Access Policy created specifically for a queue.

Example:

```
Service Bus Namespace
  |
  +----- OrderQueue
           |
           +----- Queue Policy
                   |
                   +----- Connection String
```

The application can use this connection string to access that particular queue according to the permissions assigned to the policy.

Example

Suppose we have:

```
Namespace
  |
  +----- OrderQueue
  +----- PaymentQueue
  +----- NotificationQueue
```

If we create a policy for:

OrderQueue

the application should use that queue-level policy when it only needs to work with `OrderQueue`.

6. Namespace-Level Connection String

A Service Bus Namespace can also have a Shared Access Policy.

Go to:

```
Azure Portal
  ↓
Service Bus Namespace
  ↓
Settings
  ↓
Shared access policies
```

A namespace-level policy can provide a connection string that allows access to entities within that namespace, according to the permissions granted.

For example:

```
Service Bus Namespace
|
+----- Queue 1
|
+----- Queue 2
|
+----- Queue 3
|
+----- Topic 1
|
+----- Topic 2
```

A namespace-level connection string can be used when the application needs broader access across the namespace.

7. Queue-Level vs Namespace-Level Connection String

This is an important interview topic.

Queue-Level	Namespace-Level
Created for a specific queue	Created for the Service Bus namespace
More restricted access	Broader access
Application can be limited to a specific entity	Can be used for multiple entities within the namespace, subject to permissions
Better for least privilege	Useful when broader namespace access is required
Preferred when working with one queue/topic	Useful for scenarios requiring namespace-level access

8. Example

Suppose our namespace contains:

```
MyServiceBusNamespace
|
+----- OrderQueue
|
+----- PaymentQueue
|
+----- NotificationQueue
```

Scenario 1 – Application only needs OrderQueue

Use a queue-specific policy:

```
Order Service
|
| Queue Connection String
v
OrderQueue
```

This follows the **least privilege principle**.

Scenario 2 – Application needs multiple entities

Suppose an application needs to work with:

```
OrderQueue
PaymentQueue
NotificationQueue
```

A namespace-level connection string may be appropriate if the application genuinely requires that broader access.

```
Application
|
| Namespace Connection String
v
Service Bus Namespace
|
+---- OrderQueue
+---- PaymentQueue
+---- NotificationQueue
```

9. Least Privilege Principle

The video emphasizes an important security concept:

Give an application only the permissions and access it actually needs.

Suppose an application only needs:

```
OrderQueue
```

There is no need to give it access to:

```
PaymentQueue
NotificationQueue
CustomerQueue
```

Instead:

```
Application
  |
  | Queue-specific access
  v
OrderQueue
```

This is called the **Principle of Least Privilege**.

Why is it important?

It improves security.

If the application's credentials are compromised, the attacker has access only to the resources allowed by that policy rather than the entire namespace.

10. When Should We Use Queue-Level Connection Strings?

Prefer a queue/topic-specific policy when:

- The application works with a specific queue.
- The application only needs limited access.
- You want to follow least privilege.
- You want better isolation between applications.
- You want to reduce the impact of compromised credentials.

Example:

```
Order Service
  |
  | Queue-specific connection
  v
OrderQueue
```

11. When Should We Use Namespace-Level Connection Strings?

A namespace-level connection string may be required when the application needs access at the namespace level.

One example mentioned in the video is:

Cross-entity transactions.

A cross-entity transaction is a transaction involving multiple Service Bus entities, such as multiple queues or topics within the same namespace.

For example:

```
Transaction
|
+----- Queue A
|
+----- Queue B
```

In such scenarios, namespace-level access can be required.

Important

You don't need to memorize the details of cross-entity transactions immediately.

Remember:

Specific queue/topic → Prefer entity-level access.

Operations involving multiple entities at the namespace level → Namespace-level access may be required.

12. Important Security Note

Connection strings contain credentials.

Therefore:

Never hard-code a Service Bus connection string in source code.

Avoid:

```
string connectionString = "Endpoint=...;SharedAccessKey=...";
```

inside application code or Git repositories.

Instead, use secure mechanisms such as:

- Azure Key Vault
- Managed Identity
- Environment variables
- Secure application configuration

For modern Azure applications, **Managed Identity** is generally preferable when supported because it avoids storing Service Bus credentials in the application.

13. Connection String Structure – Basic Understanding

A Service Bus connection string typically contains information similar to:

```
Endpoint=sb://<namespace>.servicebus.windows.net/;  
SharedAccessKeyName=<policy-name>;  
SharedAccessKey=<secret-key>
```

The important parts are:

Endpoint

Identifies the Service Bus namespace.

```
sb://<namespace>.servicebus.windows.net/
```

SharedAccessKeyName

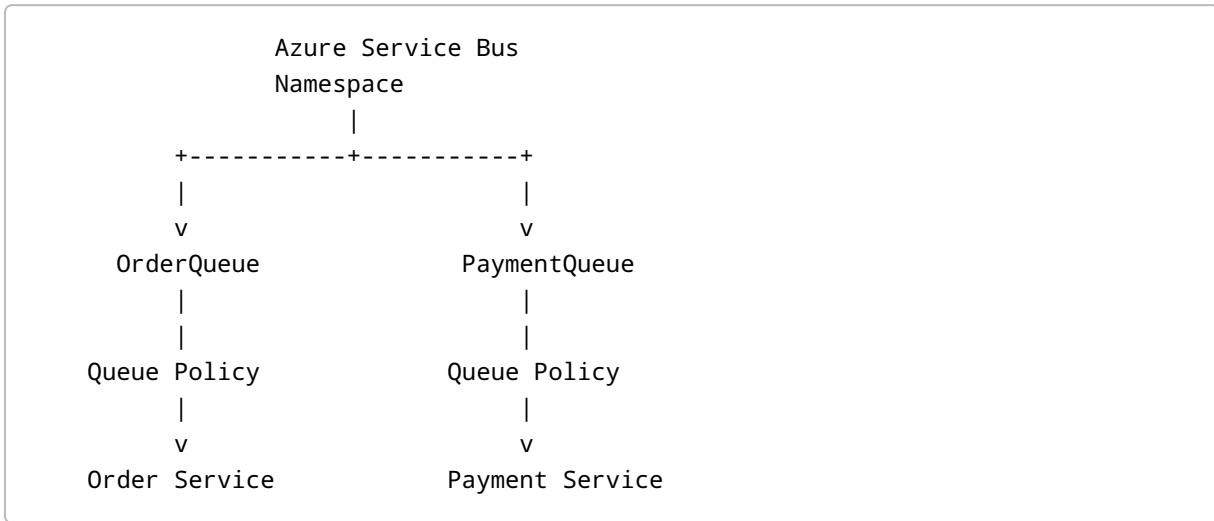
Identifies the Shared Access Policy.

SharedAccessKey

The secret key used for authentication.

Do not expose the actual connection string publicly.

14. Complete Architecture



Each service can have its own access policy.

This provides better isolation.

15. Interview Question – What is the Difference Between Queue and Namespace Connection String?

Answer

A queue-level connection string is associated with a policy for a specific Service Bus entity and is preferred when an application only needs access to that particular queue or topic. A namespace-level connection string provides broader access within the Service Bus namespace and may be required for scenarios involving multiple entities, such as certain cross-entity transactions. We should generally prefer the most restrictive access that satisfies the application's requirements.

16. Interview Question – Why Should We Prefer Queue-Level Access?

Answer

We should prefer queue-level access when the application only needs to work with a specific queue because it follows the Principle of Least Privilege. It limits the application's permissions and reduces the security impact if the credentials are compromised.

17. Interview Question – What are Primary and Secondary Keys?

Answer

Azure Service Bus provides primary and secondary keys for a Shared Access Policy. They can be used to support credential rotation. For example, we can switch an application from the primary key to the secondary key, regenerate the primary key, and then switch back after updating the application.

18. Important Points to Remember

Connection String

Used by an application to connect/authenticate with Azure Service Bus.

Shared Access Policy

Defines what access an application has.

Queue-Level Policy

Used when access is required for a specific queue.

Namespace-Level Policy

Provides broader access within the namespace according to its permissions.

Primary Key

Primary credential for the policy.

Secondary Key

Alternative credential that can be used during key rotation.

Least Privilege

Give the application only the access it actually needs.

Cross-Entity Transaction

A transaction involving multiple Service Bus entities; namespace-level access can be required for such scenarios.

19. Easy Way to Remember

Think of a **building**:

```
Service Bus Namespace = Building  
  
Queue = Individual Room  
  
Namespace-Level Access = Access to multiple rooms  
  
Queue-Level Access = Access to one specific room
```

If an employee only works in **Room A**, don't give them access to the entire building.

That's the idea of:

Principle of Least Privilege.

20. Final Revision

```
Azure Service Bus  
  |  
  +----- Namespace  
  |      |  
  |      +----- Queue  
  |      |      |  
  |      |      +----- Queue Policy  
  |      |  
  |      |
```

```
|      +----- Topic
|
+----- Namespace Policy
```

Remember:

Specific queue/topic → Entity-level policy → Least privilege

Multiple entities / namespace-level operation → Namespace-level policy

Policy → Permissions → Connection String

And the most important security rule:

Never expose or hard-code Service Bus connection strings or access keys in source code.